



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Cloud Computing: Análisis,
Diseño y Despliegue de un
Servicio Empresarial Seguro
en Microsoft Azure**



Presentado por Karima Drafi Rico
en Universidad de Burgos — Febrero 2026
Tutor: D. José Ignacio Santos Martín



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. José Ignacio Santos Martín, profesor del departamento de Ing. de Organización, área de OE.

Expone:

Que el alumno Dña. Karima Drafli Rico, con DNI 72748765h, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado “Cloud Computing: Análisis, Diseño y Despliegue de un Servicio Empresarial Seguro en Microsoft Azure”.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección de los que suscriben, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, Febrero de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. José Ignacio Santos Martín

D. José Ignacio Santos Martín

Resumen

En este Trabajo Fin de Grado se presenta un análisis, diseño y despliegue de un servicio empresarial seguro utilizando Microsoft Azure. Se describen la arquitectura cloud adoptada, los servicios implementados y las prácticas de seguridad y control de versiones aplicadas durante el desarrollo del proyecto.

Descriptores

Cloud Computing, Microsoft Azure, arquitectura basada en servicios, seguridad informática, integración continua, despliegue de aplicaciones.

Abstract

This Final Degree Project presents the analysis, design, and deployment of a secure enterprise service using Microsoft Azure. It describes the adopted cloud architecture, implemented services, and the security and version control practices applied during the project development.

Keywords

Cloud Computing, Microsoft Azure, service-based architecture, information security, continuous integration, application deployment.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	5
3.1. Cloud Computing	5
3.2. Microsoft Azure	6
3.3. Arquitectura basada en servicios	8
3.4. API REST	9
3.5. Seguridad y buenas prácticas cloud	9
3.6. DevOps y automatización	9
4. Técnicas y herramientas	11
4.1. Metodología de desarrollo	11
4.2. Lenguaje y framework backend	11
4.3. Control de versiones	12
4.4. Plataforma cloud	12
4.5. Automatización del despliegue	12
4.6. Calidad del código	13
4.7. Terraform	13
5. Aspectos relevantes del desarrollo del proyecto	17

5.1. Ciclo de vida del proyecto	17
5.2. Arquitectura del sistema	17
5.3. API REST implementada	19
5.4. Componente de análisis automático basado en reglas	20
5.5. Automatización del despliegue	22
5.6. Automatización empresarial con Logic App	22
5.7. Seguridad y gestión de secretos	22
5.8. Validación del prototipo	23
6. Trabajos relacionados	25
7. Conclusiones y Líneas de trabajo futuras	27
Bibliografía	29

Índice de figuras

5.1. Arquitectura final del prototipo desplegado en Microsoft Azure. Fuente: elaboración propia.	19
5.2. Infografía de arquitectura cloud del prototipo. Fuente: elaboración propia.	19

Índice de tablas

3.1. Modelos principales de servicio cloud	6
3.2. Responsabilidad compartida en la solución cloud	7
3.3. Servicios Azure utilizados en la solución	8
4.1. Comparación de alternativas para aplicación, datos y servicios cloud	14
4.2. Comparación de alternativas para operación y seguimiento . . .	15
4.3. Herramientas utilizadas y justificación	16
5.1. Endpoints principales de la API	20
5.2. Reglas de negocio del clasificador automático	21
5.3. Escenarios de validación de las reglas de clasificación	21
6.1. Comparativa de trabajos y soluciones relacionadas	26

1. Introducción

En los últimos años, las tecnologías cloud se han convertido en una pieza fundamental dentro de las infraestructuras empresariales modernas. Las organizaciones necesitan aplicaciones seguras, escalables y accesibles desde cualquier ubicación, permitiendo gestionar información crítica de forma eficiente y reduciendo la complejidad de mantenimiento de infraestructuras físicas.

Uno de los problemas más frecuentes en entornos empresariales es la gestión de solicitudes internas de tecnología, como peticiones de acceso, cambios de configuración, creación de entornos, altas de aplicaciones o incidencias técnicas. Muchas organizaciones continúan utilizando herramientas poco integradas como hojas de cálculo, cadenas de correos electrónicos o sistemas internos difíciles de mantener. Esto provoca problemas de trazabilidad, retrasos en la resolución y dificultades para monitorizar el estado global del servicio.

El presente Trabajo Fin de Grado propone el análisis, diseño e implementación de un sistema empresarial seguro basado en tecnologías cloud utilizando Microsoft Azure. La solución desarrollada permite registrar solicitudes operativas TI, clasificarlas automáticamente según su prioridad y tipo, generar recomendaciones operativas y gestionar la información de forma segura utilizando servicios cloud gestionados.

La arquitectura propuesta utiliza una API desarrollada en Python y Flask desplegada sobre Azure App Service, integrando además servicios como Azure Blob Storage, Azure Key Vault, identidad administrada, scripts de despliegue con Azure CLI y seguimiento de calidad mediante SonarCloud.

Durante el desarrollo del proyecto se han aplicado metodologías ágiles basadas en sprints, utilizando herramientas profesionales de gestión de tareas y control de versiones para garantizar la trazabilidad del trabajo realizado.

Como materiales de evaluación se entregan el repositorio GitHub del proyecto, la memoria y anexos en PDF, el despliegue operativo en Azure App Service, el tablero Zube utilizado para la planificación por sprints, el panel SonarCloud de calidad de código y las evidencias técnicas recogidas en la documentación del repositorio.

2. Objetivos del proyecto

El objetivo principal del proyecto es desarrollar una plataforma cloud segura en Microsoft Azure capaz de automatizar la gestión de solicitudes operativas TI mediante una arquitectura moderna basada en servicios gestionados.

Para alcanzar este objetivo principal se han definido los siguientes objetivos específicos:

- Analizar las ventajas de las plataformas cloud en entornos empresariales.
- Diseñar una arquitectura segura basada en servicios cloud.
- Implementar una API REST utilizando Python y Flask.
- Desplegar la aplicación en Microsoft Azure.
- Gestionar secretos y configuraciones sensibles mediante Azure Key Vault.
- Automatizar el despliegue de la aplicación mediante scripts reproducibles con Azure CLI.
- Gestionar el proyecto utilizando metodologías ágiles y control de versiones.
- Implementar una clasificación automática de solicitudes utilizando un componente de inteligencia artificial ligera.
- Analizar la calidad del código utilizando SonarQube/SonarCloud.

- Generar documentación técnica y funcional del sistema.

Además de los objetivos técnicos, el proyecto busca aplicar conocimientos adquiridos durante el Grado en Ingeniería Informática relacionados con ingeniería del software, seguridad informática, arquitecturas cloud y desarrollo backend.

3. Conceptos teóricos

3.1. Cloud Computing

El cloud computing es un modelo tecnológico que permite acceder a recursos informáticos bajo demanda utilizando Internet. Este modelo reduce la necesidad de mantener infraestructuras físicas propias, permite escalar recursos dinámicamente y facilita que las organizaciones consuman capacidad de cómputo, almacenamiento, red y servicios gestionados según sus necesidades reales [4].

Entre las principales ventajas del cloud computing destacan:

- Escalabilidad.
- Alta disponibilidad.
- Pago por uso.
- Reducción de costes.
- Facilidad de mantenimiento.

Estas ventajas son especialmente relevantes en proyectos de alcance empresarial, ya que permiten pasar de una infraestructura rígida a un modelo más flexible, automatizable y orientado a servicios. En el caso de este TFG, el uso de la nube permite desplegar un prototipo accesible para el tribunal sin distribuir máquinas virtuales ni depender de una instalación local compleja.

Modelos de servicio

Los servicios cloud suelen clasificarse en distintos modelos según el nivel de responsabilidad que asume el proveedor y el nivel de control que conserva el usuario. La tabla 3.1 resume los modelos principales.

Modelo	Responsabilidad principal del proveedor	Ejemplo en Azure
IaaS	Infraestructura básica: máquinas virtuales, red, discos y recursos de cómputo.	Azure Virtual Machines
PaaS	Plataforma gestionada para ejecutar aplicaciones sin administrar servidores.	Azure App Service
SaaS	Aplicación completa consumida como servicio por el usuario final.	Microsoft 365

Tabla 3.1: Modelos principales de servicio cloud

La solución desarrollada se apoya principalmente en un enfoque PaaS. Azure App Service permite ejecutar una API Flask y un portal web sin administrar directamente el sistema operativo, el balanceo interno ni la infraestructura física subyacente [9].

La tabla 3.2 resume cómo se reparte la responsabilidad entre el proveedor cloud y el equipo que desarrolla la aplicación. Esta distinción es importante porque el uso de servicios gestionados no elimina la responsabilidad de diseñar correctamente la aplicación, proteger los secretos y validar el despliegue.

Modelos de despliegue

Además de los modelos de servicio, la nube puede organizarse mediante diferentes modelos de despliegue: nube pública, privada, híbrida o multicloud. Este proyecto utiliza nube pública, ya que los recursos se despliegan en Microsoft Azure y se consumen desde una suscripción Azure for Students. Para un prototipo académico, este enfoque permite validar una arquitectura real con costes controlados y acceso remoto.

3.2. Microsoft Azure

Microsoft Azure es la plataforma cloud de Microsoft orientada al desarrollo, despliegue y administración de aplicaciones empresariales [1]. Microsoft

Ámbito	Responsabilidad del proveedor cloud	Responsabilidad del proyecto
Infraestructura física	Centros de datos, hardware, red física y energía.	Selección de región y uso responsable de recursos.
Plataforma de ejecución	Mantenimiento del servicio gestionado App Service.	Configuración de runtime, variables de entorno y publicación de la aplicación.
Datos	Disponibilidad del servicio de almacenamiento.	Modelo de datos, privacidad, validación y persistencia correcta.
Identidad y secretos	Servicios como Key Vault y Managed Identity.	Definir permisos mínimos y evitar secretos en el repositorio.
Aplicación	Componentes gestionados de plataforma.	Código Flask, autenticación, pruebas y control de calidad.

Tabla 3.2: Responsabilidad compartida en la solución cloud

describe Azure como una plataforma para crear, desplegar y gestionar soluciones en la nube, con servicios de cómputo, datos, inteligencia artificial, seguridad y operaciones [5].

Los principales servicios utilizados en este proyecto son:

- Azure App Service.
- Azure Blob Storage.
- Azure Key Vault.
- Managed Identity.
- Azure Monitor.

Servicios Azure utilizados

La tabla 3.3 relaciona los servicios Azure seleccionados con la responsabilidad concreta que cumplen dentro del prototipo.

Azure Blob Storage está orientado al almacenamiento de objetos y datos no estructurados, y permite acceder a los objetos mediante HTTP/HTTPS o bibliotecas cliente, incluidas bibliotecas para Python [15]. Azure Key

Servicio	Uso en el TFG	Justificación
App Service	Aloja la API Flask y el portal web.	Servicio PaaS adecuado para aplicaciones web y APIs REST.
Blob Storage	Guarda las solicitudes en formato JSON.	Almacenamiento sencillo, económico y apropiado para datos semiestructurados.
Key Vault	Almacena el token de autenticación.	Evita credenciales embebidas en código fuente.
Managed Identity	Permite que App Service acceda a Key Vault.	Reduce la gestión manual de secretos y claves.
Azure Monitor	Apoya la revisión de estado y logs.	Facilita observabilidad básica del despliegue.

Tabla 3.3: Servicios Azure utilizados en la solución

Vault centraliza secretos, claves y certificados, reduciendo la probabilidad de exponer información sensible en el código [8]. Las identidades administradas permiten que un recurso de Azure obtenga una identidad propia para acceder a otros servicios sin almacenar credenciales en la aplicación [14].

3.3. Arquitectura basada en servicios

Las arquitecturas basadas en servicios permiten dividir una aplicación en componentes independientes encargados de responsabilidades concretas.

Este enfoque facilita:

- Escalabilidad.
- Mantenimiento.
- Reutilización.
- Seguridad.

La arquitectura del proyecto sigue esta idea: el portal web se encarga de la interacción con el usuario, la API Flask concentra la lógica de negocio, el clasificador aplica reglas de análisis automático, Blob Storage conserva las solicitudes y Key Vault protege el secreto usado por los endpoints protegidos. Esta separación facilita explicar el sistema, probarlo por partes y evolucionarlo en futuras versiones.

3.4. API REST

Una API REST permite la comunicación entre sistemas mediante peticiones HTTP.

En este proyecto se utilizan diferentes endpoints para:

- Crear solicitudes operativas.
- Consultar solicitudes e incidencias.
- Obtener métricas.

3.5. Seguridad y buenas prácticas cloud

La seguridad es un aspecto transversal en soluciones cloud. En este TFG se han aplicado tres decisiones principales: uso de HTTPS en el despliegue, separación de secretos mediante Key Vault y autenticación Bearer para las operaciones de consulta. Estas medidas no convierten el prototipo en una solución corporativa completa, pero sí demuestran buenas prácticas básicas de diseño seguro.

Además, el diseño se ha alineado con principios del Azure Well-Architected Framework, que propone pilares como seguridad, fiabilidad, eficiencia de rendimiento, optimización de costes y excelencia operativa [10]. En el contexto del proyecto, estos principios se reflejan en el uso de servicios gestionados, despliegue reproducible, recursos de bajo coste y documentación de evidencias.

3.6. DevOps y automatización

DevOps es una metodología orientada a integrar desarrollo y operaciones con el objetivo de automatizar procesos y mejorar la calidad del software [2].

Entre las prácticas utilizadas destacan:

- Integración continua.
- Despliegue automatizado.
- Control de versiones.
- Automatización de procesos.

En la versión final del prototipo, la automatización se materializa mediante scripts de despliegue y verificación. Esta aproximación permite reproducir el despliegue sin depender de un entorno corporativo complejo y facilita mostrar al tribunal qué acciones se han ejecutado sobre Azure.

4. Técnicas y herramientas

4.1. Metodología de desarrollo

El proyecto se ha desarrollado siguiendo principios de metodologías ágiles utilizando iteraciones organizadas en sprints.

Para la gestión de tareas se ha utilizado Zube.io, permitiendo organizar historias de usuario, tareas técnicas y seguimiento del progreso.

4.2. Lenguaje y framework backend

La aplicación backend se ha desarrollado utilizando Python junto con el framework Flask.

Flask permite construir APIs REST ligeras y fáciles de mantener. Su núcleo reducido deja explícitas las decisiones sobre rutas, validación, almacenamiento y autenticación, lo que resulta adecuado para un prototipo centrado en servicios cloud y no en las funciones de un framework web completo [16].

Justificación frente a otras alternativas

La selección tecnológica se realizó atendiendo al alcance, al tipo de carga y a la facilidad para demostrar cada objetivo del TFG. La tabla 4.1 recoge las principales alternativas consideradas; no establece que una herramienta sea universalmente superior, sino cuál encaja mejor con este prototipo.

La tabla 4.2 completa el análisis con las herramientas que sostienen el despliegue, la planificación y la calidad.

4.3. Control de versiones

El control de versiones se ha realizado utilizando Git y GitHub.

Se han realizado commits frecuentes para mantener trazabilidad completa del desarrollo.

4.4. Plataforma cloud

La infraestructura cloud se basa en Microsoft Azure y en servicios gestionados documentados por Microsoft para aplicaciones web, almacenamiento, monitorización y gestión de secretos [12, 11, 13, 6].

Los servicios principales utilizados son:

- Azure App Service.
- Azure Storage.
- Azure Key Vault.
- Azure Monitor.

La tabla 4.3 resume las herramientas empleadas y el motivo de su elección dentro del proyecto.

4.5. Automatización del despliegue

El despliegue se automatiza mediante el script `scripts/deploy-azure.ps1`, que utiliza Azure CLI para configurar App Service, Storage, Key Vault, Managed Identity y publicación ZIP de la aplicación. Esta decisión permite reproducir el despliegue desde un equipo local y documentar de forma explícita cada paso técnico aplicado sobre Azure.

El script automatiza:

- Comprobación de sesión en Azure.
- Configuración de secretos y variables de entorno.
- Asignación de identidad administrada.
- Publicación de la carpeta `src/` en Azure App Service.

4.6. Calidad del código

La calidad del código se analiza mediante SonarQube/SonarCloud [17].

En este proyecto se utiliza SonarCloud como servicio gestionado compatible con repositorios GitHub. Su uso permite obtener métricas objetivas de mantenibilidad, fiabilidad, seguridad y duplicidad sin desplegar un servidor SonarQube propio. El análisis se realiza desde el panel web de SonarCloud y complementa a las pruebas unitarias.

Esta herramienta permite detectar:

- Vulnerabilidades.
- Duplicidad.
- Complejidad excesiva.
- Posibles errores.
- Deuda técnica y problemas de mantenibilidad.

Los resultados del análisis se emplean como evidencia de calidad del código y como apoyo para priorizar mejoras antes de la entrega final.

4.7. Terraform

Terraform es una herramienta Infrastructure as Code (IaC) utilizada para automatizar el aprovisionamiento de recursos cloud.

En el proyecto se conserva como definición declarativa de infraestructura Azure, permitiendo documentar recursos reproducibles, controlados y versionados mediante Git [7]. El despliegue operativo final se ha realizado con el script PowerShell incluido en el repositorio.

Necesidad	Elección	Alternativa considerada	Justificación en el proyecto
API y portal web	Flask	Django	Django aporta ORM, administración y una estructura extensa, útiles en aplicaciones con dominio relacional complejo. Flask reduce componentes no utilizados y permite concentrar el prototipo en API, reglas de negocio e integración con Azure [3].
Persistencia cloud	Azure Blob Storage con JSON	SQLite o Azure SQL	Los registros son documentos sencillos, el volumen del prototipo es reducido y no se requieren relaciones ni transacciones entre tablas. Blob Storage se integra con el objetivo cloud y evita mantener un motor de base de datos. SQLite habría sido válido para ejecución local, pero menos representativo del despliegue distribuido [18, 15].
Alojamiento	Azure App Service	Máquina virtual o contenedor administrado manualmente	App Service abstrae sistema operativo, publicación y disponibilidad básica. Una máquina virtual ofrecería más control, pero añadiría mantenimiento ajeno al proceso empresarial estudiado.
Automatización	Azure Logic Apps	Integración directa de cada sistema con la API	Logic Apps desacopla los sistemas externos del backend y hace visible la orquestación. La API conserva validación, clasificación y persistencia para evitar duplicar reglas.
Secretos	Azure Key Vault con Managed Identity	Variables o secretos almacenados manualmente	Key Vault centraliza el secreto y Managed Identity evita credenciales de acceso entre servicios dentro del código.

Tabla 4.1: Comparación de alternativas para aplicación, datos y servicios cloud

Necesidad	Elección		Alternativa considerada		Justificación en el proyecto
Despliegue	PowerShell Azure CLI	y	Pipeline completo	CI/CD	El script es reproducible, auditable y suficiente para un único entorno académico. Un pipeline sería apropiado con varios equipos y promociones entre entornos, pero aumentaría el alcance sin mejorar el caso funcional validado.
Planificación	Zube		Azure Boards, Jira o GitHub Projects		Zube ya estaba integrado con GitHub y permitió conservar tablero, historias y cinco sprints visibles. Cambiar de gestor al final habría fragmentado la trazabilidad histórica.
Calidad	SonarCloud		SonarQube auto-hospedado		SonarCloud aporta el mismo tipo de métricas sin administrar servidor, base de datos ni actualizaciones de una instancia propia.

Tabla 4.2: Comparación de alternativas para operación y seguimiento

Herramienta	Uso en el proyecto	Motivo de elección
Python y Flask	Implementación de la API REST y lógica de negocio.	Rapidez de desarrollo, claridad y ecosistema de bibliotecas.
Azure App Service	Despliegue del portal y la API.	Servicio PaaS adecuado para exponer aplicaciones web sin administrar servidores.
Azure Blob Storage	Persistencia de solicitudes en JSON.	Almacenamiento sencillo, económico y suficiente para el prototipo.
Azure Key Vault	Gestión del token de autenticación.	Separación de secretos respecto al código fuente.
Managed Identity	Acceso de App Service a Key Vault.	Evita credenciales manuales entre servicios Azure.
PowerShell y Azure CLI	Automatización del despliegue.	Reproducibilidad y trazabilidad de los pasos ejecutados.
Zube	Seguimiento de sprints y tablero Kanban.	Visibilidad del proceso y relación con GitHub.
SonarCloud	Análisis de calidad del código.	Métricas externas de mantenibilidad, fiabilidad, seguridad y duplicación.

Tabla 4.3: Herramientas utilizadas y justificación

5. Aspectos relevantes del desarrollo del proyecto

5.1. Ciclo de vida del proyecto

El proyecto se ha organizado en cinco sprints siguiendo una metodología ágil.

- Sprint 1: Análisis del problema empresarial, definición de requisitos y arquitectura inicial.
- Sprint 2: Provisionamiento de infraestructura Azure y configuración de Terraform.
- Sprint 3: Desarrollo de la API Flask, portal web y persistencia en Azure Storage.
- Sprint 4: Seguridad con Key Vault, Managed Identity, portal web y despliegue en Azure.
- Sprint 5: Pruebas, calidad SonarCloud, documentación y validación del prototipo.

5.2. Arquitectura del sistema

La arquitectura implementada sigue un modelo de servicios gestionados en Microsoft Azure. Los componentes principales son:

- **Azure App Service:** hospeda la API Flask y el portal web empresarial en Python 3.11.
- **Azure Storage:** persiste las solicitudes operativas TI en un contenedor Blob privado.
- **Azure Key Vault:** almacena el token de autenticación de la API.
- **Azure Logic Apps:** automatiza la entrada de solicitudes externas mediante un trigger HTTP.
- **Application Insights y Azure Monitor:** permiten revisar telemetría, métricas y comportamiento operativo del servicio desplegado.
- **Scripts de despliegue:** automatizan la configuración y publicación en Azure mediante Azure CLI.

El flujo principal del sistema es el siguiente:

1. Un empleado envía una solicitud operativa TI desde el portal web, mediante la API REST o a través de la Logic App.
2. Azure App Service procesa la petición con Gunicorn.
3. La API valida los datos y clasifica automáticamente la solicitud.
4. El sistema genera una recomendación operativa para el equipo de TI.
5. Los datos se almacenan en Azure Blob Storage.
6. Los secretos se obtienen desde Azure Key Vault mediante Managed Identity.
7. El endpoint de métricas y Application Insights permiten observar el estado agregado del sistema.

La figura 5.1 representa este flujo dentro de la arquitectura desplegada. Permite diferenciar los canales de entrada, la lógica ejecutada en App Service y los servicios gestionados responsables de secretos, persistencia y observabilidad.

La figura 5.2 resume visualmente los servicios cloud utilizados y su relación con el proceso empresarial automatizado.

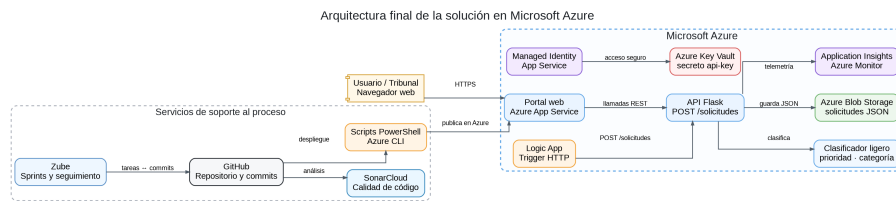


Figura 5.1: Arquitectura final del prototipo desplegado en Microsoft Azure. Fuente: elaboración propia.

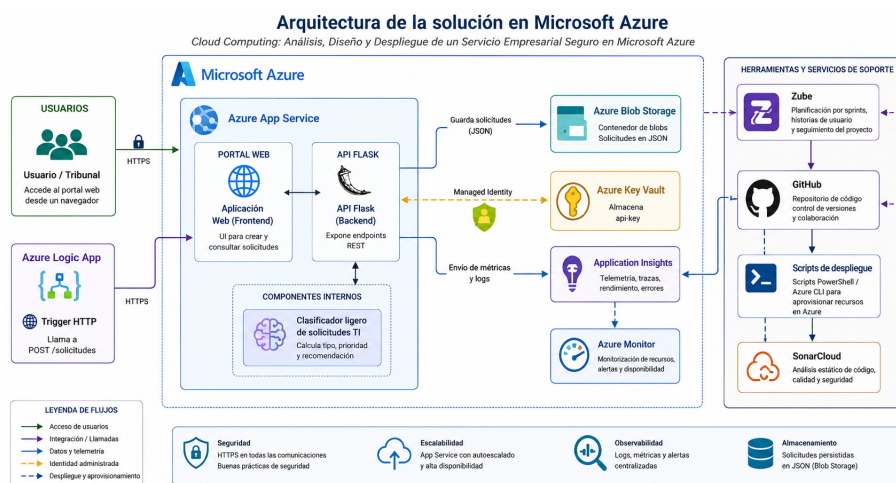


Figura 5.2: Infografía de arquitectura cloud del prototipo. Fuente: elaboración propia.

5.3. API REST implementada

La API desarrollada en `src/app.py` expone los siguientes endpoints:

- **GET /:** estado del servicio y versión.
- **GET /portal:** portal web para solicitudes operativas TI.
- **GET /health:** comprobación de salud y modo de almacenamiento.
- **POST /solicitudes:** creación de solicitudes con clasificación automática.
- **GET /solicitudes:** listado con filtros por estado, prioridad y tipo.

- GET `/metricas`: métricas agregadas del sistema.

La tabla 5.1 resume el contrato principal de la API, indicando el mecanismo de seguridad y la forma en que se ha validado cada operación.

Método	Ruta	Seguridad	Validación realizada
GET	<code>/</code>	Pública	Comprobación de estado general del servicio.
GET	<code>/portal</code>	Pública	Acceso al portal web desplegado en App Service.
GET	<code>/health</code>	Pública	Verificación de estado y modo de almacenamiento.
POST	<code>/solicitudes</code>	Pública con validación de entrada	Creación de solicitud y clasificación automática.
GET	<code>/solicitudes</code>	Token Bearer	Consulta protegida con filtros y paginación.
GET	<code>/metricas</code>	Token Bearer	Revisión de métricas agregadas del sistema.

Tabla 5.1: Endpoints principales de la API

Al crear una solicitud, el módulo `classifier.py` analiza el título y la descripción para asignar tipo de solicitud (*acceso*, *entorno*, *aplicación*, *configuración*, *incidencia*), prioridad (*baja*, *media*, *alta*), categoría (*seguridad*, *infraestructura*, *sopORTE*, *general*) y una recomendación operativa, devolviendo un valor de confianza asociado.

5.4. Componente de análisis automático basado en reglas

Se ha incorporado un clasificador ligero basado en palabras clave y reglas de negocio. Se trata de un sistema determinista de inteligencia artificial simbólica, no de un modelo de aprendizaje automático entrenado. Esta precisión es relevante: sus decisiones son trazables, repetibles y adecuadas para demostrar análisis automático sin requerir conjuntos de datos históricos ni servicios externos.

Las reglas se evalúan en un orden definido. Una prioridad o un tipo válidos indicados explícitamente por el usuario prevalecen sobre la inferencia.

En ausencia de esos valores, se calculan puntuaciones por coincidencia de términos; la categoría con mayor puntuación se selecciona y, si no hay coincidencias, se utiliza la categoría general. Cuando dos categorías empatan, se aplica el orden estable seguridad, infraestructura y soporte. El sistema produce una única clasificación y recomendación, por lo que no genera alertas paralelas ni ejecuta dos acciones incompatibles.

La tabla 5.2 resume las reglas implementadas en `src/classifier.py`.

Regla	Condición	Resultado
R-01	Prioridad manual incluida en baja, media o alta.	Se conserva la prioridad indicada con confianza máxima.
R-02	Dos o más términos críticos, o caída acompañada de un término crítico.	Prioridad alta y atención recomendada en menos de cuatro horas.
R-03	Un término crítico o dos términos de impacto medio.	Prioridad media y resolución en la siguiente ventana operativa.
R-04	No se cumple una regla de prioridad anterior.	Prioridad baja y planificación en el backlog.
R-05	Tipo manual válido o mayor puntuación de palabras del tipo.	Tipo de solicitud explícito o inferido; incidencia como valor de respaldo.
R-06	Mayor puntuación entre seguridad, infraestructura y soporte.	Categoría única; general si todas las puntuaciones son cero.
R-07	Combinación de tipo, categoría y prioridad.	Recomendación específica y plazo operativo asociado.

Tabla 5.2: Reglas de negocio del clasificador automático

Para validar el comportamiento con entradas distintas, se definieron los escenarios de la tabla 5.3. Estos casos se ejecutan automáticamente en la batería de pruebas.

Ejemplo de entrada	Tipo	Categoría	Prioridad
Intrusión y acceso no autorizado críticos	Incidencia	Seguridad	Alta
Error y fallo en servidor VPN	Incidencia	Infraestructura	Media
Ayuda para configuración manual de usuario	Configuración	Soporte	Baja
Consulta sobre calendario de vacaciones	Incidencia	General	Baja

Tabla 5.3: Escenarios de validación de las reglas de clasificación

5.5. Automatización del despliegue

Se ha implementado un proceso de despliegue reproducible mediante `scripts/deploy-azure.ps1`. Este script:

1. Comprueba la sesión de Azure CLI.
2. Obtiene la configuración de Azure Storage y Key Vault.
3. Habilita Managed Identity en Azure App Service.
4. Concede a la aplicación permisos sobre el secreto almacenado en Key Vault.
5. Configura variables de entorno y publica el código Python en Azure App Service.

5.6. Automatización empresarial con Logic App

La Logic App actúa como capa de orquestación para integrar la plataforma con otros sistemas de la empresa. En el prototipo, un sistema externo puede enviar una petición HTTP a la Logic App y esta invoca de forma controlada el endpoint `POST /solicitudes` de la API desplegada en App Service. La API conserva la responsabilidad de validar datos, clasificar la solicitud, generar la recomendación y persistir el registro en Azure Blob Storage.

Este flujo resulta útil para un escenario empresarial realista: un formulario corporativo, una herramienta de soporte, un buzón automatizado o un sistema de atención interna podrían crear solicitudes sin acceder manualmente al portal. La Logic App no sustituye al equipo de TI ni al backend, sino que automatiza la entrada de trabajo y permite que el equipo técnico consulte, priorice y supervise las solicitudes desde la plataforma.

5.7. Seguridad y gestión de secretos

La autenticación de la API utiliza un token Bearer almacenado en Azure Key Vault. La App Service accede al secreto mediante Managed Identity, evitando credenciales embebidas en el código fuente. El almacenamiento de solicitudes reside en un contenedor Blob con acceso privado.

5.8. Validación del prototipo

Las pruebas unitarias en `tests/test_api.py` verifican la creación de solicitudes, el listado, las métricas, el portal web y la clasificación automática. El prototipo puede ejecutarse localmente con almacenamiento en fichero JSON o desplegarse en Azure con persistencia en Blob Storage.

6. Trabajos relacionados

Actualmente existen numerosas soluciones empresariales orientadas a la gestión de incidencias, solicitudes internas y servicios cloud.

Herramientas como Jira Service Management, ServiceNow o Zendesk ofrecen funcionalidades avanzadas para la gestión de incidencias empresariales.

Sin embargo, estas plataformas suelen implicar costes elevados y arquitecturas complejas.

En el ámbito académico también existen diferentes trabajos relacionados con arquitecturas cloud desplegadas en plataformas como Microsoft Azure o Amazon Web Services.

El presente proyecto se diferencia por combinar:

- Arquitectura cloud.
- Seguridad empresarial.
- Automatización DevOps.
- Gestión de incidencias.
- Integración continua.

Además, el sistema desarrollado busca mantener una arquitectura sencilla y fácilmente escalable.

Frente a estas herramientas, el proyecto no busca competir funcionalmente con plataformas comerciales, sino demostrar el diseño e implementación

Solución	Enfoque	Ventajas	Limitaciones
ServiceNow	ITSM empresarial	Muy completo y configurable	Coste y complejidad altos
Jira Service Management	Gestión de tickets	Integración con desarrollo ágil	Requiere configuración avanzada
Zendesk	Soporte y atención	Interfaz madura y rápida adopción	Menor control sobre infraestructura
Proyecto desarrollado	Solicitudes TI en Azure	Código propio, cloud e IaC	Alcance académico y prototipo

Tabla 6.1: Comparativa de trabajos y soluciones relacionadas

de una solución cloud segura, automatizada y verificable dentro del alcance de un Trabajo Fin de Grado.

7. Conclusiones y Líneas de trabajo futuras

El desarrollo de este Trabajo Fin de Grado ha permitido aplicar conocimientos relacionados con ingeniería del software, arquitecturas cloud, seguridad informática y automatización de procesos.

Uno de los principales resultados obtenidos ha sido el diseño e implementación de una plataforma empresarial segura utilizando Microsoft Azure para la gestión y automatización de solicitudes operativas TI.

La solución desarrollada demuestra cómo es posible construir aplicaciones empresariales modernas utilizando servicios cloud gestionados y herramientas DevOps.

La solución implementada demuestra cómo los servicios cloud gestionados permiten automatizar procesos empresariales reduciendo la intervención manual y mejorando la trazabilidad de las operaciones.

Durante el proyecto se han utilizado tecnologías empleadas en entornos empresariales reales: Python, Flask, Terraform como documentación de infraestructura, Azure App Service, Azure Blob Storage, Azure Key Vault, Managed Identity, Azure Logic Apps, Application Insights, Azure Monitor, Azure CLI y SonarCloud.

Además, el proyecto ha permitido profundizar en aspectos relacionados con:

- Seguridad cloud.
- Gestión de secretos.

- APIs REST.
- Automatización de procesos.
- Arquitecturas escalables.
- Clasificación automática de información operativa.

El prototipo desarrollado valida los objetivos del TFG: automatización de un proceso empresarial (gestión de solicitudes operativas TI), seguridad mediante Key Vault y Managed Identity, despliegue automatizado mediante script reproducible, observabilidad básica con Application Insights y Azure Monitor, e incorporación de un componente de análisis automático para clasificación y recomendación operativa.

Como líneas futuras se plantean:

- Integrar Azure OpenAI para enriquecer la clasificación y las recomendaciones.
- Incorporar autenticación de usuarios con Azure AD.
- Crear dashboards avanzados de monitorización y alertas operativas.
- Ampliar el portal web con seguimiento de solicitudes para usuarios no técnicos.
- Migrar la persistencia a Azure SQL o Cosmos DB para consultas más complejas.
- Incorporar notificaciones automáticas por correo o Teams cuando cambie el estado de una solicitud.

Bibliografía

- [1] Azure Architecture Center, Microsoft Learn. Browse azure architectures, 2025. Consultado: marzo 2026.
- [2] Karthik Bojja. Scalable devops practices with azure: Automating secure kubernetes deployments. *IJIRSET*, 9(6), Junio 2020.
- [3] Django Software Foundation. Django at a glance, 2026. Documentación oficial, consultado junio 2026.
- [4] Microsoft. Microsoft azure: Cloud computing dictionary, 2025. Consultado: marzo 2026.
- [5] Microsoft Azure. What is azure?, 2026. Documentación oficial de Microsoft Azure, consultado junio 2026.
- [6] Microsoft Learn. Azure monitor, 2025.
- [7] Microsoft Learn. Terraform for azure, 2025.
- [8] Microsoft Learn. About azure key vault, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [9] Microsoft Learn. Azure app service overview, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [10] Microsoft Learn. Azure well-architected framework, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [11] Microsoft Learn. Quickstart: Azure blob storage client library for python, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.

- [12] Microsoft Learn. Quickstart: Deploy a python web app to azure app service, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [13] Microsoft Learn. Tutorial: Python app service using key vault and managed identity, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [14] Microsoft Learn. What are managed identities for azure resources?, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [15] Microsoft Learn. What is azure blob storage?, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [16] Pallets Projects. Flask documentation, 2026. Documentación oficial, consultado junio 2026.
- [17] SonarSource. Sonarqube documentation, 2025.
- [18] SQLite Project. Appropriate uses for sqlite, 2026. Documentación oficial, consultado junio 2026.